Gemma 4 Is Less Self-Biased Than It Looks

Sugam Panthi University of Southern Mississippi

github.com/Vein05/gemma4-paper-track Supplement: `rubber-stamp-supplement.pdf` Kaggle paper track

Abstract

Coding agents often decide whether a patch is finished by asking a model, often the one that wrote it. Studies of self-preference test whether a judge is biased toward its own outputs by comparing its scores on them with its scores on other models' outputs. The closest study controls for how hard each problem is, but not for how strong the other writers are. We ask Gemma 4 31B and eight other models to score SWE-bench patches, and we grade every patch with the hidden tests. By the usual measure, Gemma trusts its own failing patches far more than other writers' failing patches. Against writers as strong as Gemma, the gap shrinks on our main data set and is not detectable on two others. Gemma gives the top score to 61% of Kimi-K2's failing patches, the same share as its own. Most of Gemma's own failing patches are wrong fixes that pass every existing test. When Gemma is shown an existing test that asserts the behavior the issue changes, it obeys the test and breaks the correct patch. We release every agent and judge trace on GitHub and Hugging Face.

1 Introduction

In matplotlib issue 24570, Gemma 4 31B wrote a patch that fixes the reported alignment bug but also swaps the left and right cases of a shared helper. When we later asked Gemma whether this patch would pass the project's hidden tests, it gave the highest score on our 0 to 100 scale and called the patch "a simple swap" that "correctly implements the expected behavior". GLM-4.5 scored the same patch 20 because it "breaks VPacker by reversing left and right", and the hidden tests for left and right alignment fail. Coding agents make this call at the end of every task, and many make it by asking the model that wrote the patch.

The standard test of this risk compares a judge's scores on its own outputs with its scores on other models' outputs. Pombal et al. (2026) ran this test on contest programs with executed labels. Most judges favored their own programs, the preference held within each band of problem difficulty, and it was larger when the judge recognized its own output. Gemma 3 12B and 27B were the exception. The test has a second confound that difficulty bands do not remove. Errors from strong writers are harder for a verifier to detect (Zhou et al., 2025), so a judge compared with weaker writers will seem to favor itself even if it never recognizes its own work.

We ran this test on Gemma 4 31B with SWE-bench patches from five other agents. By the usual measure, Gemma scores its own failing patches 16.6 points further above judges from other model families than it scores other writers' failing patches. Gemma gives Kimi-K2's failing patches the top score exactly as often as its own, and Llama-4-Maverick's a quarter as often. Against writers at least as strong as Gemma, the gap shrinks on our held-out issues and is within noise on SWE-bench Lite and on Go, Ruby and PHP issues.

Gemma trusts strong writers' patches. Most of its own failing patches are wrong fixes: the issue's tests fail while every other existing test passes. An agent therefore needs a check that Gemma did not write. Existing tests catch regressions, so we let Gemma repair patches that break one. The repair works when the test is right. When the test asserts the old behavior that the issue asks to change, Gemma obeys the test and undoes the correct fix. Checking a patch can thus fail in two ways: the judge trusts a patch too much, or the outside evidence it is given conflicts with the issue.

We make the following contributions:

- **Measured self-preference depends on who the other writers are.** Compared only with writers as strong as itself, Gemma's extra trust in its own failing patches falls from 16.6 to 9.6 points on our held-out issues, and it is within noise on two other data sets (Section 3.1).
- **Gemma obeys outdated tests when it repairs a patch.** Shown the existing tests its patch breaks, Gemma repairs real regressions, but it breaks 16 of 23 held-back correct patches, almost all because a broken test asserts the old behavior.
- **Gemma gives the top score to most strong writers' failing patches in Python, Go, Ruby and PHP.** Most of its own are wrong fixes, and the checks we ran through Gemma missed most of them (Section 3.2).

2 Setup

Patches. The main data set is 193 held-out issues, a random sample of SWE-bench Verified issues (Jimenez et al., 2023) outside SWE-bench Verified Mini (MariusHobbhahn). We call the model that wrote a patch its writer. The 4-bit competition checkpoint of Gemma 4 31B made one attempt per issue in mini-SWE-agent v1.7.0 (SWE-agent) at temperature 0, and we graded its patches with SWE-bench's grading code. Other writers' patches are public mini-SWE-agent submissions (SWE-bench experiments) by GLM-4.5, gpt-oss-120b, Kimi-K2, Llama-4-Maverick and Qwen3-Coder, with their official labels. We removed test files from every patch and dropped very long patches.

Two more data sets test whether the result depends on our run or our language. Another team ran Gemma 4 31B on the 300 SWE-bench Lite issues, in mini-SWE-agent 2.2.1 at full precision with thinking on. We ran Gemma on 120 Go, Ruby and PHP issues from SWE-bench Multilingual (SWE-bench) in mini-SWE-agent 2.0.0 with thinking on. There the other writers are four recent models, GLM-5, Kimi-K2.5, DeepSeek-V3.2 and MiniMax-M2.5.

Judges. Nine models scored every held-out patch: Gemma 4 31B, gpt-oss-20b, Qwen3.5-9B, GLM-4.7-Flash and the five writers above. Each judge saw the issue and one patch, without its writer, and scored the chance that the patch passes the hidden tests from 0 to 100. Calls went through OpenRouter at temperature 0 with reasoning off (gpt-oss at low effort). Fewer judges scored the Lite and Go, Ruby and PHP patches, chosen before any score was read.

Measures. A judge's score on a failing patch says little alone, because some failing patches look right to everyone. We therefore measure how far a judge scores each patch above the mean of the outside judges, the judges from other model families. Extra trust is how much further above them a judge scores one group of failing patches than another group. A strong writer is one that solves at least as many issues as Gemma on the same data set. We report 95% intervals from 2,000 bootstrap resamples of issues.

SWE-bench grades a patch with two groups of tests. The issue's tests fail before the reference fix and pass after it. The other existing tests must keep passing. We call a failing patch a wrong fix when the issue's tests fail and every other test passes, and a regression when it breaks one of the other tests.

Plans. We committed a written plan, with the rule for reading its result, before computing each test. The split by writer strength and the split by failure type were chosen after we saw results, and we mark them as exploratory.

3 Results

3.1 Measured self-preference depends on who the other writers are

Against all other writers, Gemma's extra trust in its own failing patches is 16.6 points on the held-out issues and 17.8 points on Lite (Figure 1b). A standard self-preference study would report these numbers. Table 1 shows where they come from. Gemma gives 100 to 61% of its own failing held-out patches and to the same share of Kimi-K2's. For the two weakest writers the share falls to a quarter or less, and these two writers supply 59% of the other writers' failing patches.

Table 1. Gemma 4 31B's scores on failing held-out patches, by writer (exploratory). "Above outside judges" is Gemma's mean score minus the outside judges' mean on the same patches, in points.

Writer	Issues solved	Failing patches	Gemma scores 100	Above outside judges
Qwen3-Coder	70%	46	48%	+1.4
GLM-4.5	62%	61	56%	+8.9
Kimi-K2	61%	54	61%	+7.1
Gemma 4 31B	53%	85	61%	+14.1
gpt-oss-120b	30%	109	25%	-6.7
Llama-4-Maverick	27%	123	16%	-10.7

When we compare Gemma's own failing patches only with those of strong writers, the gap shrinks on both data sets where we can split the writers (Figure 1b). On Lite, Claude 4 Sonnet is the only other writer as strong as Gemma, and Gemma trusts Claude's failing patches about as much as its own (+1.7 points). On Go, Ruby and PHP every other writer is strong, and the gap is 4.2 points with an interval that includes zero. Our plan for that run had predicted a gap above zero.

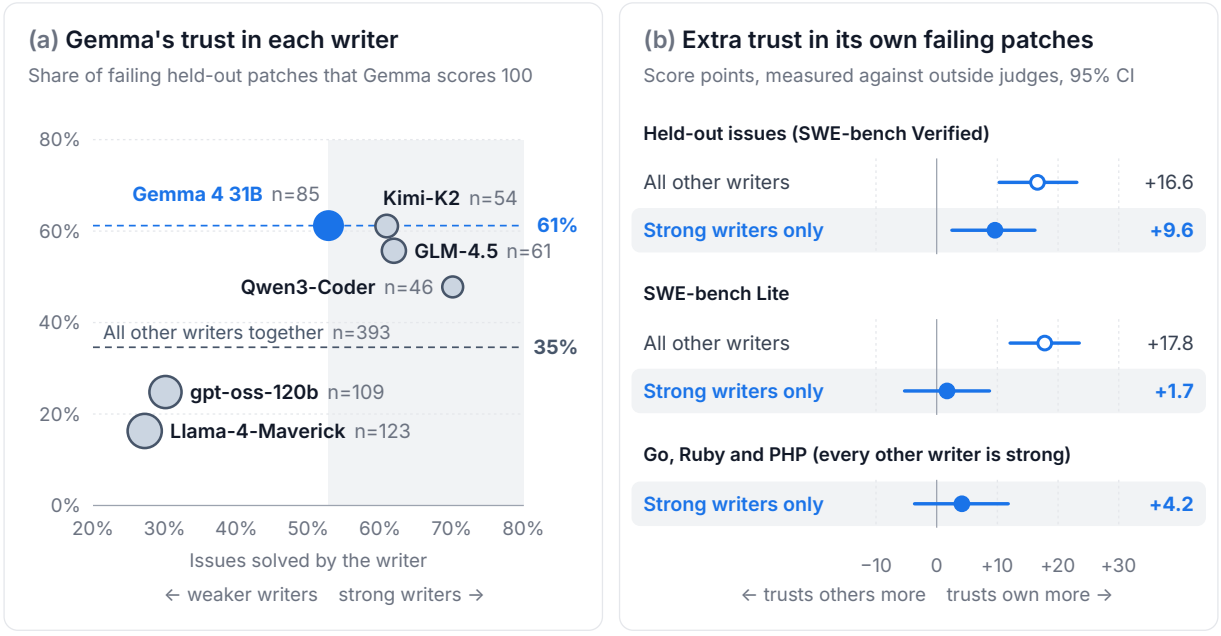


Figure 1. Gemma 4 31B trusts failing patches more when their writer is stronger. (a) Held-out issues. Shaded writers solve at least as many issues as Gemma, and dot area shows the number of failing patches. (b) Gemma's extra trust in its own failing patches, with 95% intervals, shrinks against strong writers.

A smaller gap remains on the held-out issues, and two results point to the patch content as its source. On Lite, other writers' failing patches that make nearly the same edit as Gemma's get 20 points more extra trust

than their other failing patches, although the same planned test is inconclusive on the held-out issues. If the trust came from the model family, Gemma 4 12B and 26B-A4B would share it, but they show no measurable extra trust in the 31B's failing patches (Supplement, Section D). We never varied the writer with the patch held fixed, so we cannot rule out that Gemma recognizes its own patches.

3.2 Gemma gives the top score to strong writers' wrong fixes

The trust in strong writers' failing patches holds outside our run. On Go, Ruby and PHP, Gemma gives 100 to 76% of its own failing patches and to 68% of the four other writers' failing patches. The outside judges give the same own failing patches 100 only 29% of the time. There, Gemma's score separates its own passing patches from its failing ones no better than chance.

Gemma's 100 works like a verdict. On the held-out failing patches of all six writers, it gives either 100 or a score below 50 in 92% of cases.

Most of the failing patches that get this verdict are wrong fixes. We sorted Gemma's 85 failing held-out patches by failure type (exploratory; Supplement, Section C). 55 are wrong fixes, in which every existing test still passes. 13 only break an existing test, and 17 do both. Gemma's extra trust is about the same for wrong fixes and for regressions, and the outside judges also score both kinds alike. A judge that reads the patch can see a crash, but a wrong fix looks like a fix.

Checks that Gemma runs itself miss most of the same failures (Table 2; Supplement, Section E).

Table 2. Checks run by Gemma, and what happens to its failing patches.

Check run by Gemma	What happens to Gemma's failing patches
Thinking before scoring (Mini issues)	still scores 62% of its failing patches 100, against 61% without thinking
Its own reproduction script, written while fixing the issue	the last run ends without an error in almost all failing runs that ran one
A shell with the patch applied; Gemma must write a test that fails without it	on the 52 failing held-out patches it had scored 100, still scores 80% at 100
Fine-tuning Gemma (LoRA) on labeled patches from other issues	on separate held-out and Lite issues, still accepts almost all the failing patches it had scored 100

3.3 Gemma obeys outdated tests when it repairs a patch

We gave the agent a check that uses the repository's existing tests and needs no hidden test. It picks the existing tests whose files or functions match the changed code, runs them twice before the patch and once after, and holds back the patch if a test that passed both times now fails. A held-back patch gets one more Gemma run in mini-SWE-agent. In one condition Gemma sees the broken tests and their output and is asked to make them pass without editing the test files. In the other it gets a second look (Supplement, Section F).

On the held-out issues, the broken tests let Gemma repair 14 of 26 held-back failing patches, against 6 with a second look (one-sided sign test, $p = 0.004$). On Gemma's Lite patches from the other team, the planned repeat repaired 1 of 30.

The two patch sets differ in failure type (Table 3). Most held-out failures that the check held back broke an existing test. Half of the Lite failures were wrong fixes, held back by a test outside SWE-bench's own list, and making that test pass cannot fix the issue.

Table 3. The repair on two sets of Gemma's patches, with the broken tests shown and with a second look. A

held-back patch “breaks an existing test” when it fails one of SWE-bench’s listed regression tests. Repaired counts leave out one Lite issue whose test fails at random.

	Held-out, tests	Held-out, second look	Lite, tests	Lite, second look
Held-back failing patches	26	26	30	30
Of these, break an existing test	22		15	
Repaired	14	6	1	0
Held-back correct patches	9	9	14	14
Broken by the repair	4	1	12	2
Net change in passing patches	+10	+5	−11	−2

The larger cost falls on correct patches. Some existing tests assert the old behavior that the issue asks to change. For all 14 correct Lite patches that the check held back, the reference fix fails the same check. Shown such a test, Gemma broke 12 of these 14 patches, against 2 with a second look. On xarray issue 5131 it wrote that the tests “explicitly expect” the old output, and that “the constraints of the task (making tests pass without editing them) force me to restore it”. On held-out issues the repair still helps on net: 105 of Gemma’s 180 patches pass after it, against 95 before. On Lite it hurts.

The missing step is a decision about which tests may change, which Agentless (Xia et al., 2024) leaves to the model. In an exploratory test on the saved runs, Gemma read only the issue and the broken-test report, and labeled each test as outdated or as a real regression. Skipping the repair for patches whose tests were all labeled outdated cut the broken correct Lite patches from 12 to 6 and kept 12 of the 14 held-out repairs. We chose this rule after seeing the outcomes, so it needs a fresh test.

4 Related work

Measuring self-preference. Panickssery et al. (2024) link a judge’s preference for its own summaries to how well it recognizes them. Pombal et al. (2026) asked judges whether a LiveCodeBench program passes each unit test. Most judges wrongly pass their own programs more often than other families’ programs, while Gemma 3 12B and 27B do not. Roytburg et al. (2026) compare a judge’s own wrong answers with other models’ wrong answers to the same question, and much of the measured self-preference goes away. Their correction concerns the judge. Ours concerns the other writers, and it applies even when the judge and the problems stay the same.

Verifying strong writers. On math, knowledge and reasoning tasks, Zhou et al. (2025) find that weak generators’ errors are easier to detect than strong ones’. Table 1 shows the same pattern for repository patches. Khullar et al. (2026) find that one model rates a SWE-bench patch as more correct when it is framed as its own earlier answer. Our judges see no authorship cue.

Tests as a check. In test-based program repair, test suites are often too weak, so repair tools produce many incorrect patches that pass them (Xiong et al., 2017). Our wrong fixes are this class. When code changes, some tests must change too, and Shang et al. (2026) find that coding agents identify those tests with an F1 below 50%. ImpossibleBench (Zhong et al., 2025) edits SWE-bench tests to conflict with the issue and tells the agent to put the issue first. Our conflicts come from the repositories’ own tests, and our note gave no order. Gemma chose the test.

5 Limitations

The writer-strength and failure-type splits are exploratory, and the strong group holds one to four writers per data set. The own-patch gap against strong writers stays above zero on the held-out issues. The Go, Ruby and PHP run also changes the agent version, thinking and the language, so it supports the writer account without isolating it. In Gemma's held-out agent runs, the repository's Python environment was not active, so Gemma often could not import the code. This lowers its resolve rate and may produce more crash patches, which the repair fixes most easily. The repair's harm rests on 23 correct patches, and each Lite repair condition ran once.

6 Conclusion

A judge that seems to favor its own patches may partly be favoring strong writers. For Gemma 4 31B, the measured preference for its own failing patches shrinks, or is no longer detectable, once the comparison writers are as strong as Gemma, so self-preference studies should report who the other writers were. Gemma gives most strong agents' failing patches the top score, and the checks we ran through Gemma left most of its own failing patches undetected. Existing tests detect regressions, but when a test conflicts with the issue, Gemma follows the test. An agent built on Gemma should not let Gemma's score decide that a patch is finished, and it should not let Gemma decide alone which test to believe.

7 Data, code and compute

The GitHub repository holds the paper, supplement, code, analysis plans and judge answers. The Hugging Face dataset holds the agent runs and judge prompts. Running `./reproduce.sh main` regenerates every table in this paper from the released data, without a GPU. The project used about 90 GPU-hours. At RunPod on-demand prices this compute costs about $150.

References

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press and Karthik Narasimhan. 2023. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* https://arxiv.org/abs/2310.06770. ICLR 2024 (arXiv comment, https://openreview.net/forum?id=VTF8yNQM66).

Dipika Khullar, Jack Hopkins, Rowan Wang and Fabien Roger. 2026. *Self-Attribution Bias: When AI Monitors Go Easy on Themselves*. https://arxiv.org/abs/2603.04582.

MariusHobbhahn. *swe-bench-verified-mini*. https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini. Hugging Face dataset; revision b316c349; accessed 2026-10-05.

Arjun Panickssery, Samuel R. Bowman and Shi Feng. 2024. *LLM Evaluators Recognize and Favor Their Own Generations*. https://arxiv.org/abs/2404.13076.

José Pombal, Ricardo Rei and André F. T. Martins. 2026. *Self-Preference Bias in Rubric-Based Evaluation of Large Language Models*. https://arxiv.org/abs/2604.06996.

Dani Roytburg, Matthew Bozoukov, Matthew Nguyen, Jou Barzdukas, Mackenzie Puig-Hall and Narmeen Oozeer. 2026. *Are LLM Evaluators Really Narcissists? Sanity Checking Self-Preference Evaluations*. https://arxiv.org/abs/2601.22548. ICML 2026 Main.

Ye Shang, Quanjun Zhang, Haichuan Hu, Chunrong Fang, Liang Xiao and Zhenyu Chen. 2026. *Breaking, Stale, or Missing? Benchmarking Coding Agents on Project-Level Test Evolution*. https://arxiv.org/abs/2605.06125.

SWE-agent. *mini-SWE-agent*. https://github.com/SWE-agent/mini-swe-agent. GitHub repository; accessed 2026-10-05.

SWE-bench. *SWE-bench experiments*. https://github.com/SWE-bench/experiments. GitHub repository; patches from the swe-bench-submissions bucket; revision 40f164d5; accessed 2026-10-05.

SWE-bench. *SWE-bench Multilingual*. https://huggingface.co/datasets/SWE-bench/SWE-bench_Multilingual. Hugging Face dataset; revision 846e647b; accessed 2026-11-07.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn and Lingming Zhang. 2024. *Agentless: Demystifying LLM-based Software Engineering Agents*. https://arxiv.org/abs/2407.01489.

Yingfei Xiong, Xinyuan Liu, Muhan Zeng, Lu Zhang and Gang Huang. 2017. *Identifying Patch Correctness in Test-Based Program Repair*. https://arxiv.org/abs/1706.09120. ICSE 2018 (arXiv comment).

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan and Ofir Press. 2024. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. https://arxiv.org/abs/2405.15793.

Ziqian Zhong, Aditi Raghunathan and Nicholas Carlini. 2025. *ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases*. https://arxiv.org/abs/2510.20270. ICLR 2026 (proceedings page https://iclr.cc/virtual/2026/poster/10009390, https://openreview.net/forum?id=SeO4vyAj7E).

Yefan Zhou, Austin Xu, Yilun Zhou, Janvijay Singh, Jiang Gui and Shafiq Joty. 2025. *Variation in Verification: Understanding Verification Dynamics in Large Language Models*. https://arxiv.org/abs/2509.17995. ICLR 2026 (arXiv comment).